

skaler tool - AI Video Platform

Diseño del Sistema

Índice

1. Objetivo
2. Arquitectura general
3. Componentes principales
4. Modelo de dominio
5. Relación entre Agent, Pipeline, Tool y Provider
6. Producción de un video
7. Audio / TTS
8. Transcripción
9. Visual Planning
10. Render
11. Storage
12. Procesamiento asíncrono
13. Principios arquitectónicos
14. Pendientes de diseño
15. Regla principal del sistema
16. Entidades de Base de Datos — Definición
17. Flujo de Preparación y Generación del Video

arquitectura backend

Implementacion backend

1. Objetivo

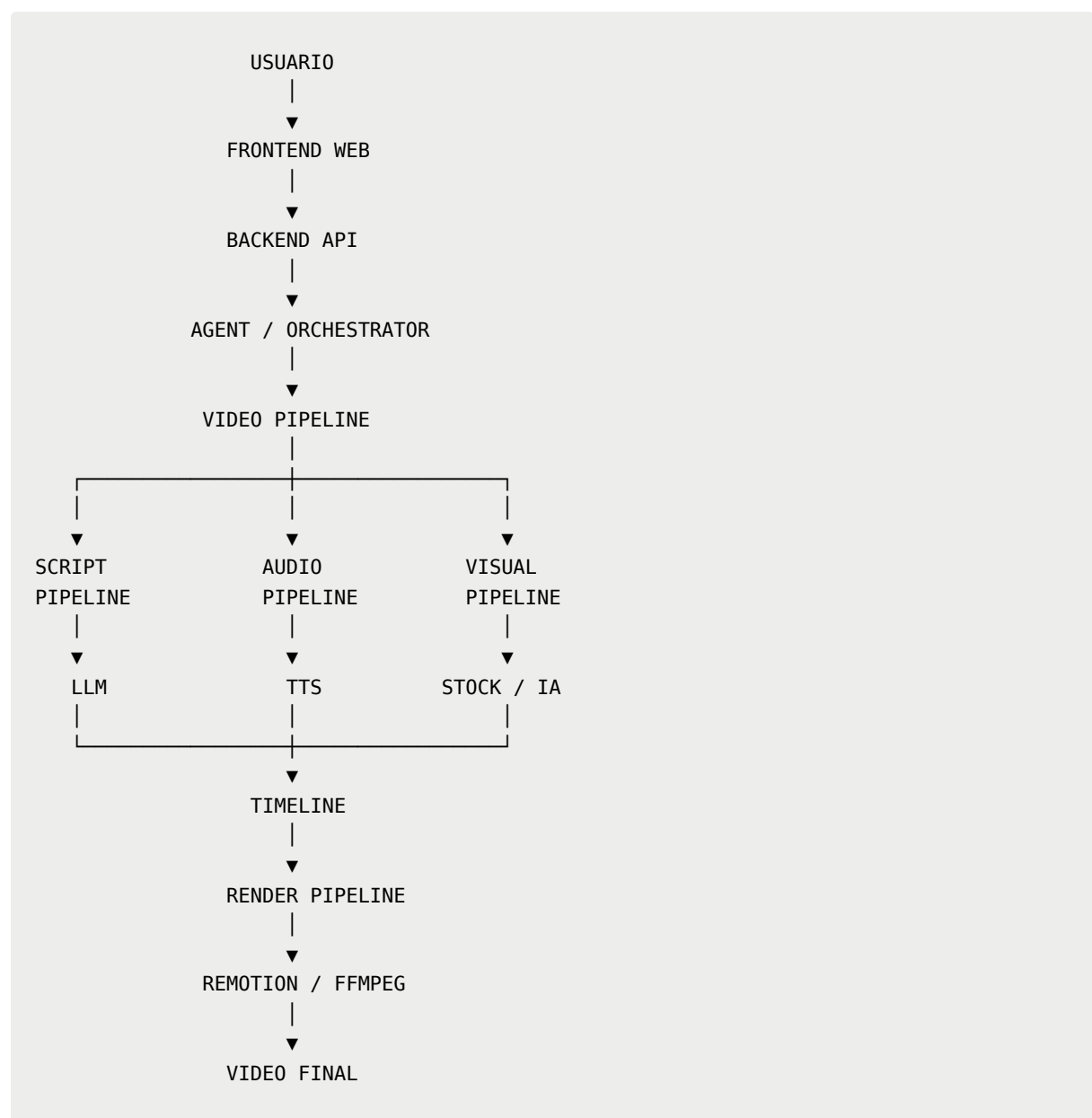
Construir una plataforma web que permita crear videos mediante interacción con un agente de IA.

El usuario podrá describir el video que desea generar y el sistema será responsable de transformar esa solicitud en un proceso de producción compuesto por diferentes etapas:

Guion → Voz → Transcripción → Timeline → Visuales → Assets → Render → Video final

La arquitectura toma como referencia el sistema actual de producción de videos documentado en `LEEME_SISTEMA_COMPLETO.txt`, cuyo flujo ya contempla TTS, Whisper, análisis visual, stock, overlays, Remotion y FFmpeg.

2. Arquitectura general



3. Componentes principales

Frontend

Interfaz web mediante la cual el usuario interactúa con la plataforma.

Responsabilidades

- Chat con el agente.
 - Creación y administración de proyectos.
 - Visualización del guion.
 - Visualización de escenas.
 - Visualización de assets.
 - Estado de generación.
 - Preview del video.
 - Acceso al video final.
-

Backend

Núcleo de la aplicación y responsable de toda la lógica de negocio.

Responsabilidades

- Exponer la API.
- Gestionar usuarios.
- Gestionar proyectos.
- Ejecutar agentes.
- Ejecutar pipelines.
- Ejecutar tools.
- Gestionar proveedores externos.
- Gestionar jobs.
- Persistir información.
- Administrar assets.

- Gestionar el proceso de render.
-

Agent / Orchestrator

Componente encargado de interpretar la intención del usuario y determinar qué acciones debe realizar el sistema.

El Agent no debería comunicarse directamente con cada API externa. Su función es decidir y coordinar.

Ejemplo

```
Usuario:
"Quiero un video de 15 minutos sobre la historia de River"

↓

Video Agent

↓

Determina:
- generar guion
- generar escenas
- generar voz
- obtener visuales
- generar overlays
- renderizar
```

Nota

El concepto de Agent es una abstracción propuesta para la nueva plataforma. El sistema actual documentado no utiliza un Agent como entidad independiente, aunque OpenAI cumple una función de dirección visual.

Pipeline

Proceso compuesto por varias etapas ordenadas.

Una Pipeline coordina diferentes Tools para conseguir un resultado.

Pipeline actual de referencia

```
Preparación
↓
```

```
TTS
↓
Whisper
↓
Timeline
↓
OpenAI
↓
Stock
↓
Unión + Audio
↓
Overlays
↓
Render
```

Este flujo está documentado en el sistema actual.

Pipelines propuestas

```
Script Pipeline
Voice Pipeline
Transcription Pipeline
Visual Pipeline
Timeline Pipeline
Render Pipeline
```

Tool

Capacidad individual que puede ejecutar una operación concreta.

La Tool representa **qué puede hacer el sistema**.

Tools iniciales

```
generate_script
generate_voice
transcribe_audio
search_stock
generate_image
generate_video
generate_overlay
render_video
```

Estas capacidades se derivan de las operaciones que actualmente realizan los scripts y herramientas del sistema existente.

Provider

Servicio concreto que implementa una Tool.

La separación permite cambiar proveedores sin modificar la lógica de negocio de la Tool.

Ejemplo

```
generate_voice
  ↓
ElevenLabs
```

Providers identificados en el sistema actual

```
ElevenLabs → TTS
Gemini → fallback TTS
Pexels → Stock
Pixabay → Stock
Whisper → Transcripción
Remotion → Overlays
FFmpeg → Composición / Render
OpenAI → Análisis visual
```

Estas responsabilidades están definidas en la documentación existente.

4. Modelo de dominio

User

Usuario de la plataforma.

Se relaciona con los proyectos que crea y administra.

VideoProject

Unidad principal de trabajo.

Representa un video en producción y agrupa todos los recursos y procesos asociados.

```
VideoProject
├─ Conversation
├─ Script
├─ Scenes
├─ Assets
├─ Timeline
├─ Jobs
└─ Final Video
```

El sistema actual trabaja alrededor de archivos como `guion.txt`, `titulo.txt` y `audio.mp3`; la nueva plataforma reemplaza ese enfoque por un proyecto persistente.

Conversation

Conversación entre el usuario y el Agent relacionada con un proyecto.

Permite conservar el contexto utilizado para generar o modificar el contenido.

Message

Mensaje individual dentro de una Conversation.

Puede representar:

- solicitud del usuario
 - respuesta del Agent
 - resultado de una acción
 - información de estado
-

Script

Contenido narrativo del video.

Define principalmente **qué se dice**.

El sistema actual establece explícitamente que el guion determina lo que se dice.

Scene

Unidad individual de contenido dentro del video.

Cada Scene representa una parte de la narración y puede contener:

```
narration
start
end
visual
audio
overlay
```

La Scene es una abstracción nueva para convertir el timeline y los recursos actuales en una estructura de dominio.

Asset

Recurso utilizado o producido durante la generación del video.

Tipos posibles

```
Audio
Image
Video
Overlay
Subtitle
```

El sistema actual genera diferentes recursos intermedios como audio, clips, overlays y videos.

Timeline

Representación temporal de todo el contenido del video.

Define cuándo comienza y termina cada elemento.

```
00:00 — Scene 1
00:18 — Scene 2
00:42 — Scene 3
01:15 — Scene 4
...
15:00 — END
```

El sistema actual genera `timeline_base.json` y posteriormente resuelve los tiempos utilizando la información obtenida por Whisper.

Job

Representa una ejecución de procesamiento.

Permite ejecutar procesos largos de manera asíncrona y conocer su estado.

Estados posibles

```
QUEUED
RUNNING
SCRIPT_DONE
AUDIO_DONE
VISUALS_DONE
RENDERING
COMPLETED
FAILED
```

ToolExecution

Registro de una ejecución específica de una Tool.

Permite conocer:

```
tool
provider
input
output
status
duration
error
cost
```

Su objetivo principal es facilitar debugging, reintentos, trazabilidad y control de costos.

5. Relación entre Agent, Pipeline, Tool y Provider

Esta es una de las reglas arquitectónicas principales:

```
USER
↓
AGENT
↓
PIPELINE
↓
```

```
TOOL
↓
PROVIDER
↓
EXTERNAL API
```

Ejemplo

```
Usuario
↓
Video Agent
↓
Voice Pipeline
↓
generate_voice
↓
ElevenLabs
↓
audio.mp3
```

Otro ejemplo:

```
Usuario
↓
Video Agent
↓
Visual Pipeline
↓
search_stock
↓
Pexels
↓
video clip
```

6. Producción de un video

Flujo completo

```
Usuario
↓
Chat
↓
Video Agent
↓
Crear VideoProject
```

```
↓
Generar Script
↓
Dividir en Scenes
↓
Generar Voice
↓
Transcribir Audio
↓
Construir Timeline
↓
Planificar Visuales
↓
Generar / Obtener Assets
↓
Generar Overlays
↓
Construir Timeline Final
↓
Render
↓
Video Final
```

7. Audio / TTS

Proveedor principal:

ElevenLabs

Fallback:

Gemini TTS

El sistema actual define explícitamente:

1. audio existente
2. ElevenLabs
3. Gemini como fallback

8. Transcripción

La transcripción tiene una función más amplia que convertir audio en texto.

Whisper / Faster-Whisper permite obtener tiempos reales de las palabras y segmentos.

Esto permite sincronizar:

- escenas
 - overlays
 - elementos visuales
 - narración
-

9. Visual Planning

El sistema necesita determinar qué debe verse durante cada parte del video.

Actualmente OpenAI cumple esta función:

```
Guion
↓
OpenAI
↓
stock_keywords
overlay
```

En la nueva plataforma el Visual Planner deberá poder seleccionar:

```
Stock
Imagen generada por IA
Video generado por IA
Overlay
```

10. Render

El Render Pipeline recibe:

```
Timeline
+
Assets
```

y genera el video final.

Actualmente:

```
Remotion
↓
Overlays
```

```
FFmpeg
↓
Unión
↓
Audio
↓
Overlays
↓
Video final
```

11. Storage

Los archivos generados deben almacenarse y asociarse al VideoProject.

Recursos

```
Audio
Images
Video Clips
Overlays
Intermediate Files
Final Video
```

El sistema actual maneja estos recursos mediante archivos y carpetas locales.

En la plataforma nueva deberán abstraerse mediante **Assets** y almacenamiento persistente.

12. Procesamiento asíncrono

La generación de un video completo puede implicar muchas operaciones y no debe depender de una única request HTTP.

```
POST /videos
↓
Create Job
↓
202 Accepted
↓
Background Processing
↓
Pipeline
↓
Final Video
```

El frontend debe poder consultar el estado del Job y mostrar el progreso.

13. Principios arquitectónicos

1. Separación de responsabilidades

Cada componente debe tener una responsabilidad concreta.

```
Agent → decide  
Pipeline → coordina  
Tool → ejecuta capacidad  
Provider → implementa capacidad  
Render → compone
```

2. Providers intercambiables

Una Tool no debe estar acoplada permanentemente a un único proveedor.

```
generate_voice  
├─ ElevenLabs  
└─ Gemini
```

3. Procesamiento asíncrono

Las operaciones largas deben ejecutarse mediante Jobs.

4. Trazabilidad

Cada ejecución debe poder conocerse y auditarse.

5. Control de costos

Las APIs externas deben monitorearse y limitarse.

El sistema actual ya contempla límites de gasto de OpenAI y desactiva la recarga automática cuando se desea controlar costos.

6. Revisión humana

La automatización no elimina la necesidad de revisar el resultado final.

El sistema actual considera obligatoria la revisión visual del stock antes del render final.

14. Pendientes de diseño

Estos puntos todavía no quedan definidos únicamente con el documento fuente:

```
[ ] Modelo definitivo de Base de Datos
[ ] Arquitectura exacta de Agents
[ ] Contrato de Tools
[ ] Modelo de Pipelines
[ ] Sistema de Jobs / Queue
[ ] Object Storage
[ ] Provider de generación de imágenes
[ ] Provider de generación de video IA
[ ] Autenticación
[ ] Sistema de usuarios / permisos
[ ] Costos y límites por usuario
[ ] Estrategia de reintentos
[ ] Observabilidad / logs
```

15. Regla principal del sistema

El usuario solicita. El Agent decide. La Pipeline coordina. Las Tools ejecutan. Los Providers producen. Los Assets se organizan en una Timeline. El Renderer construye el video final.

03. Entidades de Base de Datos — Definición

Esta sección contiene la definición iterativa de las entidades principales del sistema.

Las entidades se definirán de manera incremental, validando cada una antes de avanzar con la siguiente.

VideoProject

Descripción

Entidad raíz del sistema. Representa un video que el usuario está creando y agrupa todos los elementos, recursos y procesos asociados a su producción.

Campos

```
id
user_id
title
description
target_duration
status
created_at
updated_at
```

Status

```
DRAFT
REVIEW
COMPLETED
FAILED
```

Significado de los estados

- **DRAFT:** el proyecto está siendo creado o editado.
- **REVIEW:** el video fue generado y requiere revisión del usuario.
- **COMPLETED:** el proyecto fue revisado y finalizado.
- **FAILED:** ocurrió un error que impidió completar el proceso.

Los estados relacionados con el procesamiento técnico de una generación no pertenecen a `VideoProject` ; serán responsabilidad de `Job` .

Relación

```
User 1 ———— N VideoProject
```

Script

Descripción

Representa el contenido narrativo asociado a un `VideoProject`.

Es la fuente principal que determina qué se va a decir durante el video. El sistema actual parte de un `guion.txt` como entrada principal del proceso de producción.

Campos

```
id
video_project_id
content JSONB
created_at
updated_at
```

Content

El contenido del script se almacenará como `JSONB` para permitir una estructura flexible.

Ejemplo conceptual:

```
{
  "title": "La historia de River Plate",
  "language": "es",
  "duration": 900,
  "sections": [
    {
      "title": "Fundación",
      "narration": "River Plate fue fundado..."
    }
  ]
}
```

La estructura definitiva del JSON todavía queda abierta a definición.

Relación

VideoProject 1 — 1 Script

Scene

Descripción

Representa una unidad narrativa y audiovisual individual dentro de un `Script`.

Una escena contiene una parte del contenido del guion que posteriormente será utilizada para definir los recursos necesarios para producir esa parte del video.

El sistema actual trabaja posteriormente sobre segmentos, tiempos y elementos visuales derivados del guion y de la transcripción.

Campos

```
id
script_id
order
content JSONB
created_at
updated_at
```

Content

`content` contiene la información específica de la escena.

Ejemplo conceptual:

```
{
  "narration": "River Plate fue fundado el 25 de mayo de 1901.",
  "visual": {
    "type": "stock",
    "prompt": "historic football club in Argentina"
  },
  "overlay": null
}
```

La estructura definitiva del JSON queda abierta a medida que definamos el proceso de producción.

Order

Indica la posición de la escena dentro del guion.

```
1
2
3
4
...
```

Nota sobre los tiempos

Por el momento, `start_time` y `end_time` no forman parte de `Scene`.

La información temporal será definida posteriormente mediante la entidad `Timeline`.

Relación

```
Script 1 — N Scene
```

Asset

Descripción

Representa un recurso utilizado o generado durante la producción de un video.

El sistema actual trabaja con recursos como audio, clips, overlays y archivos de video durante las diferentes etapas de producción.

Campos

```
id
video_project_id
scene_id
type
metadata JSONB
created_at
updated_at
```

Type

Representa el tipo de recurso.

Tipos iniciales:

```
AUDIO
IMAGE
VIDEO
OVERLAY
```

Otros tipos podrán agregarse posteriormente si son necesarios.

Scene

`scene_id` será nullable.

Esto permite que un asset pueda pertenecer:

- específicamente a una escena;
- al proyecto completo.

Ejemplos de assets asociados a una escena:

```
Scene 3
├─ audio
├─ image
└─ overlay
```

Ejemplos de assets generales del proyecto:

```
VideoProject
├─ audio completo
└─ video final
```

Metadata

`metadata` almacenará información adicional específica del asset mediante `JSONB`.

Ejemplo para audio:

```
{
  "provider": "elevenlabs",
```

```
"duration": 18.5
}
```

Ejemplo para imagen:

```
{
  "provider": "openai",
  "width": 1920,
  "height": 1080
}
```

Storage

Por el momento, `Asset` no define una URL ni una ubicación física del archivo.

La decisión sobre el almacenamiento de archivos será definida posteriormente mediante la arquitectura de `Storage`.

El objetivo es separar:

```
PostgreSQL
→ información y metadata del Asset

Object Storage
→ archivo físico
```

Relaciones

```
VideoProject 1 — N Asset
Scene 1 — N Asset
```

`scene_id` puede ser `NULL` para assets pertenecientes al proyecto pero no a una escena específica.

Timeline

Descripción

Representa la estructura temporal del video.

Mientras que `Scene` define el contenido narrativo, `Timeline` define **cuándo ocurre cada elemento dentro del video**.

Ejemplo:

```
00:00 — Scene 1
00:18 — Scene 2
00:42 — Scene 3
01:15 — Scene 4
...
15:00 — FIN
```

Campos

```
Timeline
-----
id
video_project_id
content JSONB
created_at
updated_at
```

Content

Almacena la estructura temporal completa del video.

Ejemplo conceptual:

```
{
  "duration": 900,
  "scenes": [
    {
      "scene_id": "scene-01",
      "start": 0,
      "end": 18.5
    },
    {
      "scene_id": "scene-02",
      "start": 18.5,
      "end": 42.2
    }
  ]
}
```

La estructura podrá evolucionar posteriormente para incluir otros elementos temporales.

Relación

```
VideoProject 1 ——— 1 Timeline
```

Por ahora, cada proyecto tendrá una única timeline.

Job

Descripción

Representa una tarea o proceso que el sistema debe ejecutar sobre un proyecto.

Un **Job** no representa una función ni una Tool. Representa **el trabajo solicitado al sistema**.

Ejemplos:

```
Generar video
Renderizar video
Regenerar escena
Generar audio
```

Flujo conceptual:

```
Usuario solicita una acción
    ↓
  Job
    ↓
se ejecuta
    ↓
COMPLETED / FAILED
```

Campos

```
Job
-----
```

```
id
video_project_id
type
status
progress
error
created_at
started_at
finished_at
```

Type

Indica qué trabajo representa.

Ejemplos iniciales:

```
VIDEO_GENERATION
RENDER
REGENERATE_SCENE
```

Los tipos podrán ampliarse según aparezcan nuevas necesidades.

Status

```
QUEUED
RUNNING
COMPLETED
FAILED
```

Progress

Representa el avance de la ejecución.

```
0 → 100
```

Relación

```
VideoProject 1 — N Job
```

Un mismo proyecto puede tener múltiples Jobs.

Agent

Descripción

Representa al agente de IA encargado de interpretar una solicitud y decidir qué acciones realizar utilizando las herramientas disponibles.

El **Agent** coordina y razona sobre el proceso, pero las capacidades concretas se ejecutan mediante **Tools**.

Conceptualmente:

```
Usuario
  ↓
Agent
  ↓
decide qué hacer
  ↓
Tools
```

Campos

```
Agent
-----
id
name
description
model
configuration JSONB
created_at
updated_at
```

Model

Define el modelo utilizado por el agente.

Ejemplo conceptual:

```
GPT-...
Claude ...
Gemini ...
```

Configuration

Permite almacenar configuración adicional del agente.

Ejemplo:

```
{
  "temperature": 0.7,
  "max_steps": 10
}
```

La estructura definitiva de esta configuración se definirá posteriormente.

Provider

Descripción

Representa un servicio externo utilizado por el sistema.

A diferencia de **Tool**, que se define en código, un **Provider** contiene información configurable que necesita persistirse.

Ejemplos:

```
OpenAI
ElevenLabs
Pexels
Runway
```

Relación conceptual:

```
Tool
↓
Provider
↓
Servicio externo
```

Ejemplo:

```
generate_voice
↓
```

ElevenLabs

Campos

```
Provider
-----
id
name
slug
api_key
configuration JSONB
is_active
created_at
updated_at
```

Name

Nombre del proveedor.

Ejemplos:

```
OpenAI
ElevenLabs
Pexels
```

Slug

Identificador interno único utilizado por el sistema.

```
openai
elevenlabs
pexels
```

API Key

Credencial utilizada para consumir el servicio externo.

La API key debe almacenarse **cifrada**, ya que el sistema necesita recuperarla para realizar llamadas al proveedor.

Configuration

Configuración adicional específica del proveedor.

Ejemplo:

```
{
  "base_url": "...",
  "timeout": 300
}
```

Is Active

Permite habilitar o deshabilitar un proveedor sin eliminarlo.

```
true
false
```

Tool

Tool no será una entidad de base de datos.

Las Tools representan capacidades del sistema y estarán definidas directamente en código TypeScript.

Ejemplos:

```
generate_script
generate_voice
generate_image
generate_video
render_video
```

Conceptualmente:

```
Agent
↓
Tool Registry
↓
Tool
↓
Provider
↓
API externa
```

ToolExecution

Descripción

Representa el registro de una ejecución concreta de una Tool.

Su objetivo es proporcionar **trazabilidad y observabilidad** sobre las herramientas utilizadas durante un `Job`.

Ejemplo:

```
Job: Generar video
├─ generate_script → COMPLETED
├─ generate_voice → COMPLETED
├─ generate_visuals → COMPLETED
└─ render_video → COMPLETED
```

Campos

```
ToolExecution
-----
id
job_id
provider_id
tool_name
status
input JSONB
output JSONB
error
started_at
finished_at
created_at
```

Relaciones

```
Job 1 ——— N ToolExecution

Provider 1 ——— N ToolExecution
```

`tool_name` identifica la Tool definida dentro del registry de código.

Estado de la entidad

`ToolExecution` queda como **opcional** para el MVP.

No es necesaria para ejecutar el sistema, pero permite:

- registrar qué Tools se ejecutaron;
 - conocer qué Provider se utilizó;
 - registrar errores;
 - medir tiempos de ejecución;
 - facilitar debugging;
 - tener trazabilidad de los procesos.
-

Storage

`Storage` **no será una entidad de base de datos.**

Representa una decisión de infraestructura sobre dónde se almacenarán físicamente los archivos generados.

Posibles implementaciones:

```
Supabase Storage  
Amazon S3  
Cloudflare R2  
Google Cloud Storage  
Azure Blob Storage
```

Conceptualmente, el backend utilizará un servicio de almacenamiento:

```
StorageService  
|  
├─ upload()  
├─ download()  
├─ delete()  
└─ getUrl()
```

La implementación concreta podrá cambiar sin afectar las entidades del dominio.

Actualización de Asset

Para vincular un **Asset** con su archivo físico, agregamos el campo:

```
storage_key
```

Por lo tanto, la entidad queda:

```
Asset
-----
id
video_project_id
scene_id
type
storage_key
metadata JSONB
created_at
updated_at
```

Storage Key

Representa la ubicación o identificador del archivo dentro del Object Storage.

Ejemplos conceptuales:

```
projects/{project_id}/audio/audio.mp3
```

```
projects/{project_id}/scenes/{scene_id}/image.png
```

La separación queda:

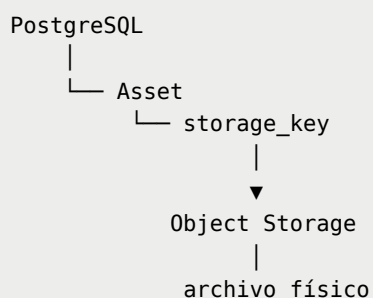
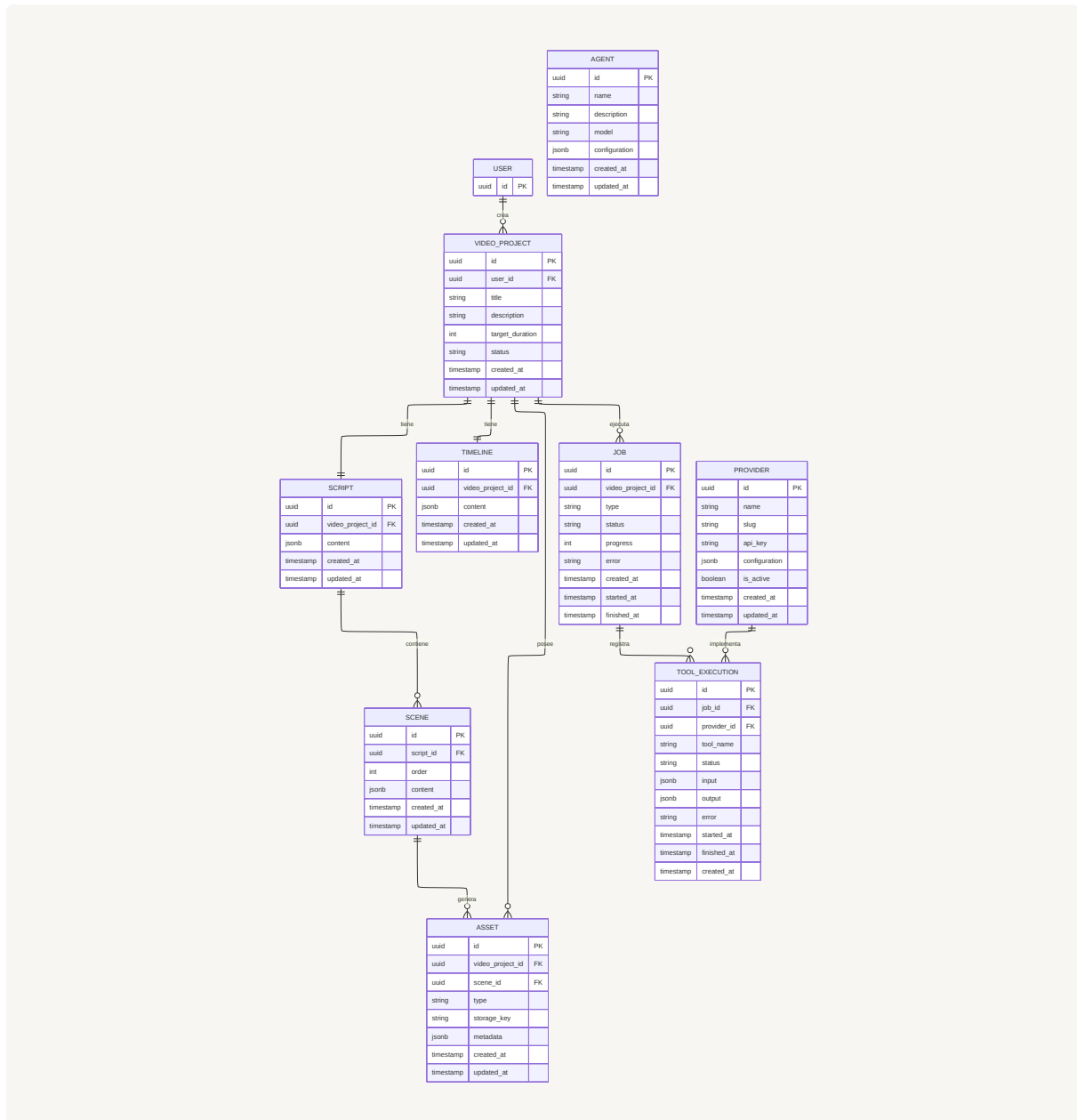


Diagrama bd · MERMAID

#



04. Flujo de Preparación y Generación del Video

Objetivo

Esta etapa define el flujo general del sistema desde la interacción inicial del usuario hasta la generación del video.

El proceso se divide en dos grandes etapas:

1. **Preparación del contenido**
2. **Producción del video**

La preparación permite recopilar y estructurar la información necesaria antes de crear el proyecto y comenzar la producción.

4.1 Inicio de la interacción

El usuario inicia una conversación mediante el chat.

```
graph TD; Usuario --> Chat; Chat --> Agent;
```

Usuario
↓
Chat
↓
Agent

El usuario puede proporcionar información de diferentes formas:

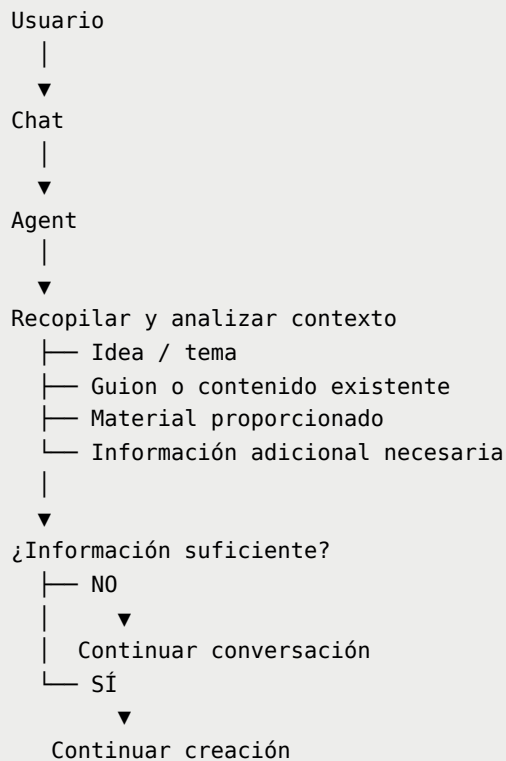
- Una idea.
- Un tema.
- Un prompt.
- Un guion.
- Un audio.
- Una transcripción.
- Archivos o contenido adicional.

El objetivo inicial del Agent es interpretar la solicitud y recopilar el contexto necesario para la creación del video.

4.2 Preparación y validación del contenido

Antes de crear un **VideoProject**, el Agent analiza la información proporcionada durante la conversación.

Conceptualmente:

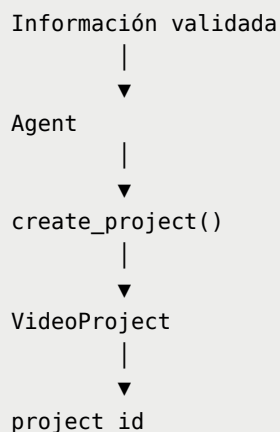


Durante esta etapa todavía no es necesario crear un `VideoProject`.

El chat puede utilizarse para desarrollar la idea, analizar un guion, procesar una transcripción o recopilar la información necesaria.

4.3 Creación del VideoProject

Una vez que el Agent considera que existe suficiente información para comenzar la creación, ejecuta la Tool correspondiente.

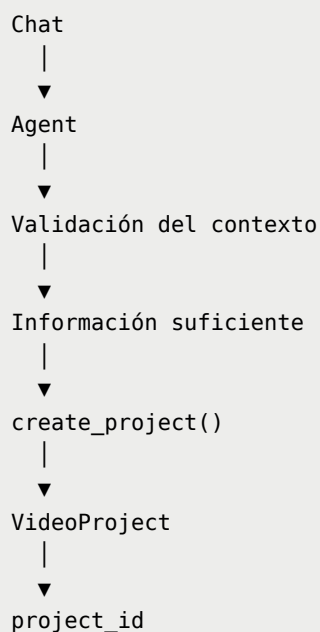


A partir de este momento existe un proyecto persistente dentro del sistema.

El `project_id` permitirá asociar posteriormente:

- Script.
- Scenes.
- Assets.
- Timeline.
- Jobs.
- Video final.

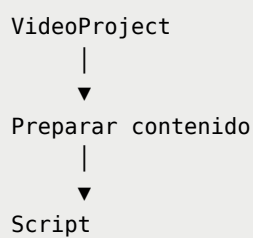
El flujo queda:



4.4 Generación y preparación del contenido

Una vez creado el proyecto, el sistema comienza a trabajar sobre el contenido.

El contenido puede provenir directamente del usuario o ser generado y adaptado por el Agent.



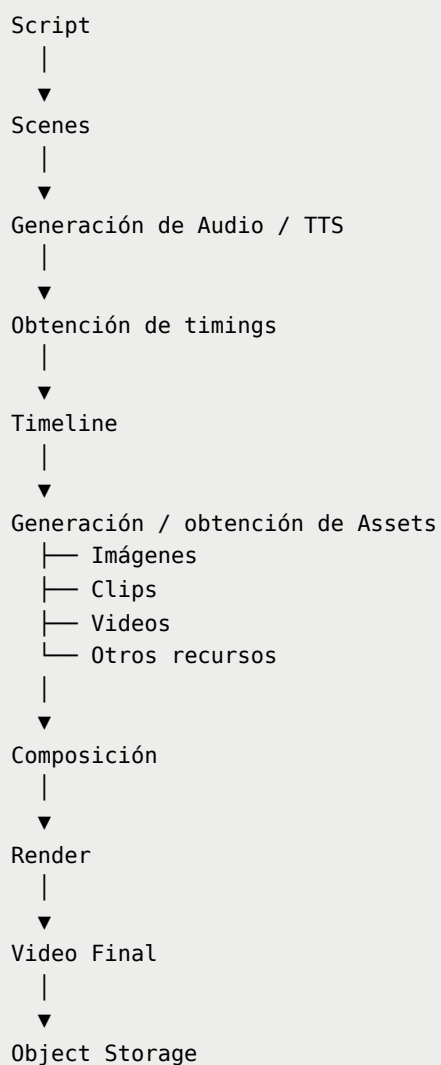


El resultado de esta etapa será un **Script** estructurado y dividido en **Scenes**.

Cada **Scene** representará una unidad sobre la cual posteriormente podrán generarse los distintos elementos necesarios para la producción.

4.5 Pipeline de producción

Una vez que el contenido está preparado, comienza la pipeline de producción.

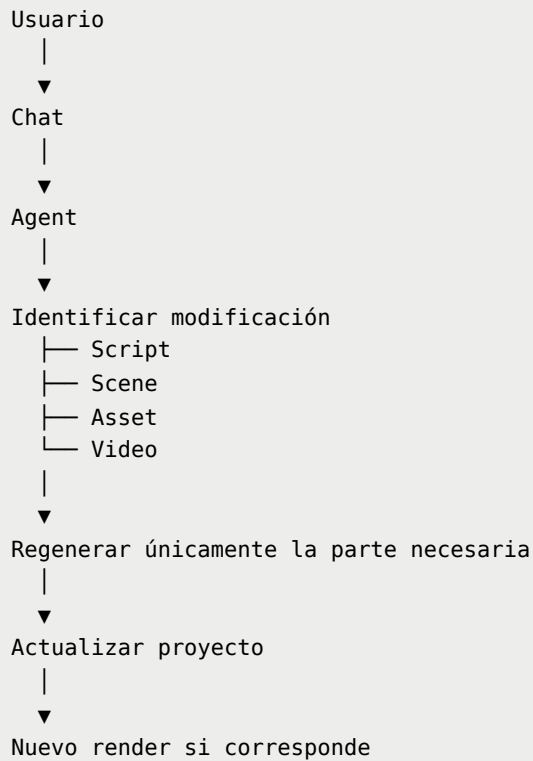


Los archivos generados durante el proceso podrán almacenarse y asociarse al proyecto mediante **Asset**.

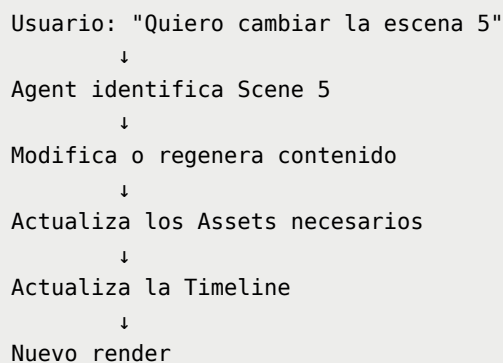
4.6 Iteración sobre el contenido

Una vez generado el contenido, el usuario podrá volver a interactuar con el proyecto mediante el chat.

El Agent deberá interpretar qué parte del proyecto se desea modificar.



Ejemplo:



4.7 Flujo general

